

Rapport de projet

Démonstrateurs



Auteur

Montois Léo

Professeur Superviseur

Wilbaut Christophe

Année Universitaire

2025-2026

Sommaire

Introduction.....	4
1.1 – Contexte du projet : Médiation scientifique en informatique	4
1.2 – Problématique : Rendre l'invisible tangible	4
1.3 – Présentation de la solution	5
1.4 - Objectifs du rapport.....	5
Dépôt du code source	5
Architecture technique.....	5
2.1 – Le choix du Stack ?.....	5
2.2 – Architecture de données	6
2.3 – Gestion des performances.....	6
2.4 – Visuels indispensables	7
Démonstrateur 1 : Sudoku	7
3.1 – Théorie des CSP (Contraintes Satisfaction Problems).....	7
3.2 – Approche algorithmiques	8
3.2.1 – Le Backtracking naïf (Force Brute)	8
3.2.2 – L'heuristique MRV (Minimum Remaining Values)	8
3.2.3 – La modélisation par coloration de graphe (DSATUR).....	9
3.3 – Implémentation technique et interactivité	9
3.4 – Analyse comparative de performance.....	10
3.5 – Visuel indispensable	10
Démonstrateur 2 : Boids.....	11
4.1 – Théorie de l'émergence et modèles de Reynolds.....	11
4.2 – Modélisation : Les trois règles fondamentales	11
4.3 – Optimisation technique et gestion des limites (Le défi du $O(n^2)$)\	12
4.4 – Analyse expérimentale et réglages du comportement	12
4.5 – Visuel indispensable	13
Démonstrateur 3 : Réseaux Sociaux	14
5.1 – Théorie des graphes et modélisation sociale	14
5.2 – Algorithme et modèles de propagation.....	14
5.2.1 – Diffusion déterministe vs Probabiliste	14
5.2.2 – Le modèle SIR (Susceptible, Infected, Recovered).....	14
5.3 – Interactivité et visualisation "Force-Directed"	15
5.4 – Analyse de l'influence de la structure.....	15
5.5 – Visuel indispensable	15
Analyse Critique et Perspectives	16

6.1 – Bilan technique et limites rencontrées.....	16
6.2 – Ouvertures et améliorations futures	17
Conclusion	17
Source bibliographique.....	17
Annexes	18
9.1 – Annexe A : Guide d'installation et Manuel utilisateur	18
9.1.1 – Prérequis techniques	18
9.1.2 – Installation des dépendances	18
9.1.3 – Scripts de lancement automatisés.....	18
9.1.4 – Guide d'utilisation rapide	18
9.1.5 – Dépannage (Troubleshooting)	19
9.2 – Annexe B : Glossaire Technique	19
9.2.1 – Algorithmique et Théorie des Graphes	19
9.2.2 – Systèmes Multi-Agents et Simulation.....	20
9.2.3 – Développement Web et Rendu Graphique	20
9.3 – Annexe C : Code	21

Introduction

Sujet : Conception et développement de démonstrateurs algorithmiques interactifs pour la médiation scientifique.

1.1 – Contexte du projet : Médiation scientifique en informatique

Ce projet s'inscrit dans une démarche de vulgarisation et de médiation scientifique initiée par l'équipe pédagogique de la Licence d'Informatique. Dans un monde où les algorithmes régissent une part influente de l'activité humaine de la logistique industrielle aux recommandations sociales leur compréhension par le grand public reste superficielle. L'informatique est souvent perçue comme une discipline de "boîte noire" où les processus de décision sont invisibles.

L'enjeu des journées portes ouvertes ou des événements tels que des salons étudiants sont de briser cette barrière. Il s'agit de démontrer que l'informatique n'est pas seulement une affaire de programmation, mais une science de la modélisation, de la résolution de problèmes et de l'étude des comportements émergents. Pour ce faire, le support visuel est indispensable.

1.2 – Problématique : Rendre l'invisible tangible

La problématique centrale de ce travail est la suivante : Comment transformer des concepts abstraits d'algorithmes et de théorie des graphes en une expérience visuelle, interactive et immédiate ?

Expliquer le concept de « backtracking » ou de « satisfaction de contraintes » par le biais d'équations ou de code brut est inefficace pour un public non spécialiste. La solution réside dans la simulation interactive. Cependant, créer de tels outils impose des défis majeurs :

1- L'immédiateté : Le démonstrateur doit être utilisable sur un stand, sans formation préalable.

2- La fluidité : La simulation doit être visuellement convaincante (60 images par secondes).

3- La pertinence pédagogique : La visualisation doit mettre en lumière la différence d'efficacité entre deux méthodes de résolution (ex : force brute vs heuristique).

1.3 – Présentation de la solution

Pour répondre à ces enjeux, nous avons développé une plateforme intégrée regroupant trois univers de démonstration :

- Le sudoku : Illustration des problèmes combinatoires et de la propagation de contraintes.
- Les Boids : Modélisation de l'émergence collective et des systèmes multi-agents.
- Les Réseaux Sociaux : Étude de la topologie des graphes et des phénomènes de diffusion virale.

1.4 - Objectifs du rapport

Ce rapport documente le cycle complet de réalisation du projet. Nous y détaillerons les choix technologiques effectués pour garantir la portabilité du logiciel, les modèles mathématiques sous-jacents à chaque démonstrateur, ainsi qu'une analyse comparative des performances expérimentales obtenues.

Dépôt du code source

Ajout du dépôt Github : <https://github.com/Ningo/Demonstrateurs>

Architecture technique

2.1 – Le choix du Stack ?

Le cahier des charges imposait une contrainte forte : le système doit fonctionner de manière autonome, sans installation complexe sur les postes de présentation.

L'écosystème Web (HTML/CSS/JS) :

Nous avons opté pour une architecture d'application web moderne. L'avantage est triple : une portabilité totale (tout navigateur web suffit), une interface utilisateur riche et une facilité de déploiement (un simple lien URL ou un dossier local).

Vue.js : Framework de réactivité :

Pour la gestion de l'interface et de l'état de l'application, notre choix s'est porté sur Vue.js. Contrairement à une manipulation manuelle du DOM (HTML/XML) qui est

source d'erreurs et de lenteurs, Vue.js utilise un système de réactivité basé sur des proxies.

- Modularité : Chaque démonstrateur est conçu comme un composant isolé (un fichier vue pour chaque démonstrateur).

- Liaison de données (Data-binding) : Les curseurs (sliders) permettant de régler la vitesse des Boids ou les probabilités de diffusion sont liés directement aux variables du moteur de calcul. Cela permet une interaction "en temps réel" sans rechargement de page.

HTML5 Canvas : Le moteur de rendu :

Pour la partie simulation (Boids et Réseaux), nous utilisons l'API Canvas.

Contrairement au SVG qui traite chaque élément comme un objet du DOM, le Canvas traite l'écran comme un tableau de pixels. Cela permet d'animer des centaines de Boids ou de nœuds réseau avec une consommation CPU minimale, en utilisant le thread graphique du navigateur.

2.2 – Architecture de données

L'application suit un patron de conception découplé. Le moteur algorithmique est séparé de la vue :

- 1- L'état (State) : Contient les positions des Boids, la grille de Sudoku ou la liste d'adjacence du graphe.
- 2- Le moteur de calcul : Des classes JavaScript pures qui transforment l'état T en état $T + 1$.
- 3- La boucle de rendu : Utilise *requestAnimationFrame* pour synchroniser le dessin sur le Canvas avec le taux de rafraîchissement de l'écran.

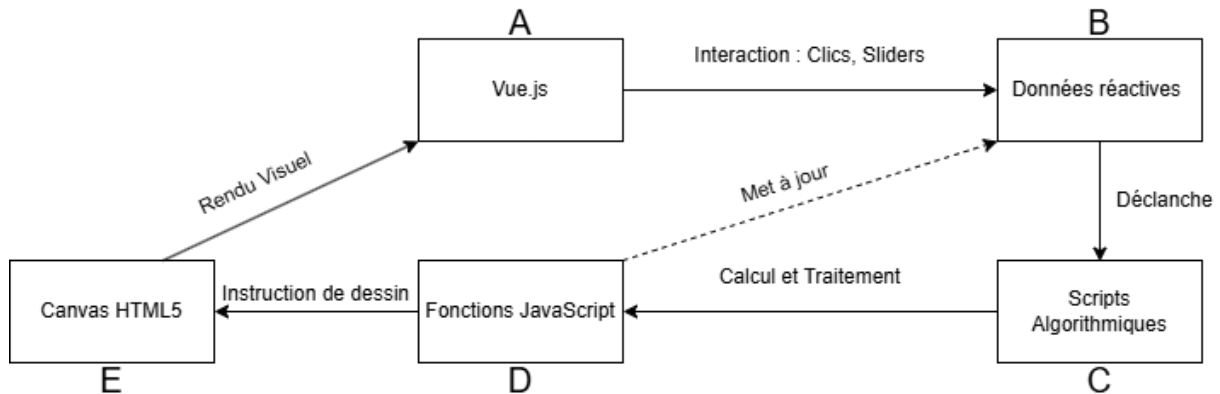
2.3 – Gestion des performances

Une problématique majeure en JavaScript est le blocage du thread principal. Si un algorithme de Sudoku effectue 100 000 tests, l'interface utilisateur "gèle".

Pour résoudre cela, nous avons implémenté :

- L'asynchronisme contrôlé : Utilisation de *async/await* et de délais artificiels (*sleep*) pour permettre au navigateur de redessiner l'interface entre deux étapes de calcul. C'est ce qui permet au public de "voir" le backtracking s'opérer.

- Optimisation des calculs vectoriels : Utilisation de tableaux typés (Float32Array)



pour accélérer les calculs mathématiques dans la simulation de Boids.

2.4 – Visuels indispensables

Figure : Architecture logicielle du démonstrateur.

Ce schéma représente la séparation des préoccupations au sein de notre application :

1. La Couche Interface (Vue.js) : Elle capte les interactions de l'utilisateur (curseurs de réglage, choix d'algorithme) et les traduit en un état réactif.
2. Le State (Données) : Il agit comme la source unique de vérité. Toute modification de l'interface met à jour cet état, qui est immédiatement observé par le moteur de calcul.
3. Le Moteur Algorithmique (Logique métier) : Découplé de l'interface, il traite les données mathématiques (CSP, règles de Reynolds, propagation de graphe). Il est asynchrone pour ne pas bloquer le thread principal.
4. Le Rendu (Canvas) : Il reçoit les instructions de dessin provenant des algorithmes et effectue le rendu graphique final à 60 FPS, garantissant une expérience fluide malgré la complexité des calculs en arrière-plan.

Démonstrateur 1 : Sudoku

3.1 – Théorie des CSP (Contraintes Satisfaction Problems)

Le Sudoku, bien que perçu comme un simple divertissement, constitue en informatique un cas d'étude fondamental de la classe des Problèmes de Satisfaction de Contraintes (CSP). Un CSP est défini par un triplet (X, D, C) :

- X (Variables) : L'ensemble des 81 cases de la grille (9x9).
- D (Domaines) : Pour chaque case vide, l'ensemble des valeurs possibles est l'intervalle d'entiers [1,9]
- C (Contraintes) : Les règles du jeu. Elles sont au nombre de trois : l'unicité par ligne, l'unicité par colonne et l'unicité au sein du bloc 3 x 3 (appelé région).

Dans le cadre de la médiation scientifique, le Sudoku est l'outil idéal pour illustrer ce qu'est une "explosion combinatoire". Si nous remplissons une grille vide de manière totalement aléatoire, il y aurait 9^{81} combinaisons possibles, un nombre dépassant largement le nombre d'atomes dans l'univers observable. La résolution informatique nécessite donc des stratégies d'élagage pour explorer cet espace de recherche.

3.2 – Approche algorithmiques

3.2.1 – Le Backtracking naïf (Force Brute)

L'algorithme de retour sur trace (Backtracking) est l'approche de base. Il s'agit d'un algorithme récursif qui explore l'arbre de décision en profondeur.

- 1- Il sélectionne la première case vide disponible (ordre de lecture standard).
- 2- Il tente d'y placer le chiffre '1'. Si la contrainte est respectée, il passe à la case suivante.
- 3- Il tente d'y placer le chiffre '1'. Si la contrainte est respectée, il passe à la case suivante.
- 4- **Analyse de complexité** : La complexité théorique est en $O(d^n)$ où d est la taille du domaine (9) et n le nombre de cases vides. Visuellement, le démonstrateur montre au public que cet algorithme "tâtonne". On voit les chiffres défiler et revenir en arrière de manière systématique, ce qui est très efficace pour illustrer le concept de recherche exhaustive.

3.2.2 – L'heuristique MRV (Minimum Remaining Values)

Pour optimiser la recherche, nous avons implémenté l'heuristique MRV. Au lieu de choisir les cases dans l'ordre, l'algorithme calcule pour chaque case vide son "degré

de liberté" (le nombre de chiffres valides restants). Il sélectionne alors la case possédant le domaine le plus restreint.

- Logique : En traitant les cases les plus contraintes en premier, on réduit le facteur de branchement de l'arbre de recherche. Cela permet de détecter les échecs beaucoup plus tôt dans la pile récursive.

3.2.3 – La modélisation par coloration de graphe (DSATUR)

Une originalité de ce projet est la transformation du Sudoku en un problème de théorie des graphes.

- Construction du graphe : Chaque case est un sommet (81 sommets). Une arête est créée entre deux sommets s'ils appartiennent à la même unité (ligne, colonne ou bloc).
- Coloration : Résoudre le Sudoku revient à colorier le graphe avec 9 couleurs sans que deux sommets adjacents n'aient la même couleur.
- Algorithme DSATUR : Cet algorithme priorise le coloriage des sommets ayant le plus grand "degré de saturation" (nombre de couleurs différentes déjà utilisées par ses voisins). Cette approche est particulièrement puissante pour montrer au public comment une abstraction mathématique permet de résoudre un problème concret de manière élégante.

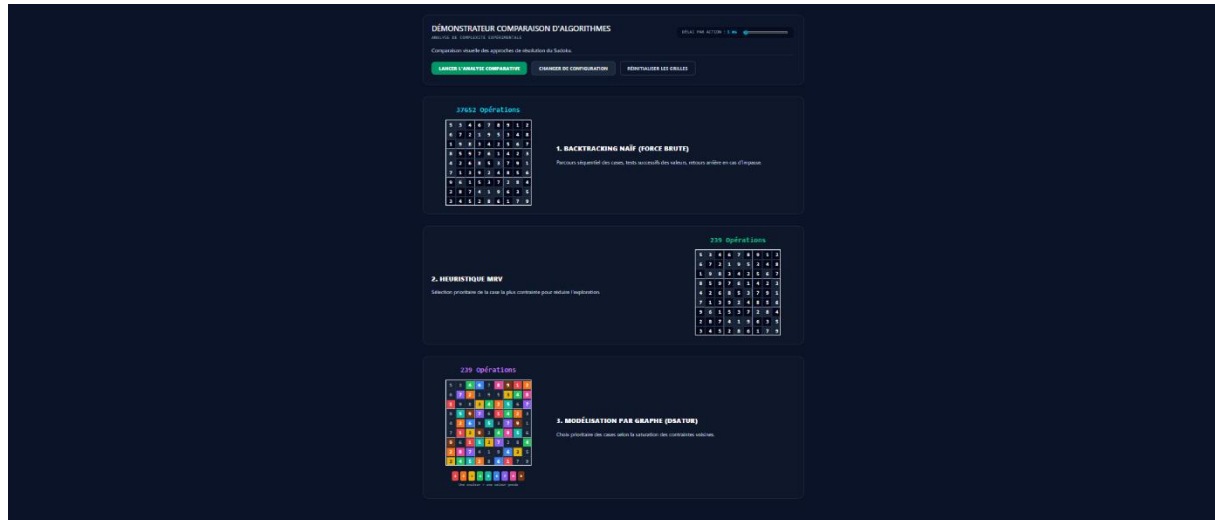
3.3 – Implémentation technique et interactivité

Le défi de l'implémentation a été de rendre l'algorithme "visible". En JavaScript, une fonction récursive s'exécute en quelques millisecondes, ce qui est invisible pour l'œil humain. Nous avons utilisé des fonctions asynchrones (*async/await*) couplées à un délai ajustable par l'utilisateur via un curseur de vitesse. À chaque modification de la grille en mémoire, le moteur de rendu Canvas redessine la cellule concernée.

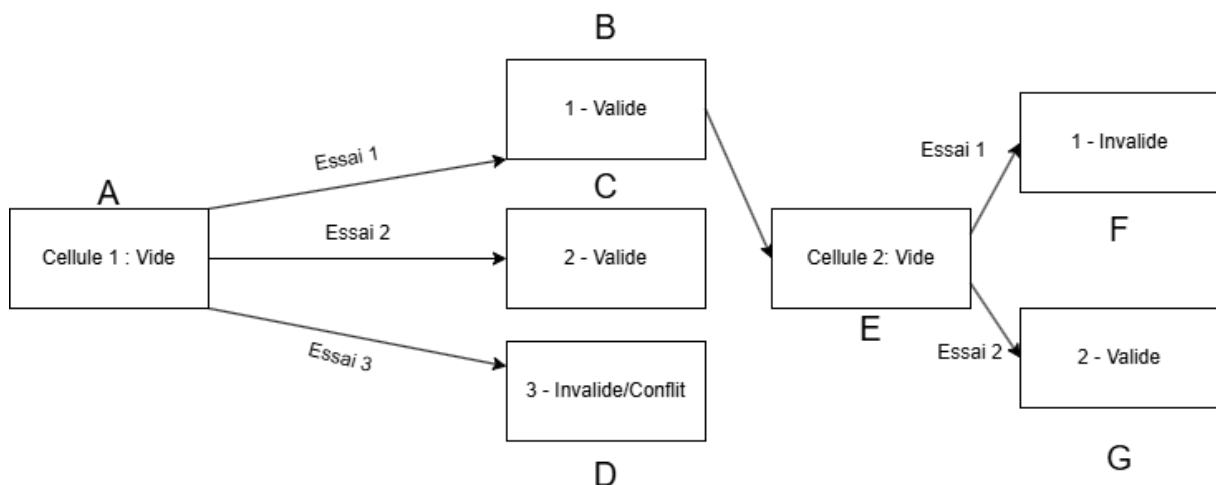
- Indicateurs : Un compteur d'itérations ("Nombre d'essais") est affiché en temps réel, permettant de voir la machine effectuer des milliers de tests par seconde.

3.4 – Analyse comparative de performance

Les tests effectués montrent des disparités flagrantes. Sur une grille de niveau "Expert" :



- Backtracking naïf : Peut nécessiter plus de 100 000 étapes.
- MRV / DSATUR : Résolvent la même grille en moins de 500 étapes. Cette comparaison est le point d'orgue de la démonstration sur stand : elle prouve



que "réfléchir" avant de calculer (via des heuristiques) est plus performant que la simple puissance de calcul.

3.5 – Visuel indispensable

L'interface est épurée pour éviter la charge cognitive. Les contrôles sont placés de manière à ce que l'utilisateur puisse modifier la vitesse de résolution sans quitter la simulation des yeux. Sur cette capture on peut également voir que le nombre

d'opérations est nettement supérieur pour l'algorithme de backtracking que les deux autres.

Ce schéma illustre le fonctionnement récursif de l'algorithme de résolution en backtracking naïf.

1. Nœuds (Cercles) : Représentent l'état de la grille à une profondeur donnée.
2. Branches : Représentent les différentes tentatives de chiffres (de 1 à 9) pour une case donnée.
3. Pruning (Élagage) : Les nœuds colorés en orange indiquent une violation des contraintes (doublet en ligne, colonne ou bloc). L'algorithme détecte l'erreur immédiatement et arrête l'exploration de cette branche : c'est le mécanisme de Backtracking.
4. Progression : Les nœuds en vert montrent un cheminement valide. L'algorithme ne remonte en arrière que lorsqu'il atteint un cul-de-sac (impasse), ce qui permet d'explorer l'espace des solutions de manière exhaustive mais optimisée.

Démonstrateur 2 : Boids

4.1 – Théorie de l'émergence et modèles de Reynolds

Le second démonstrateur s'éloigne de la logique pure pour explorer les systèmes complexes. Le modèle des "Boids", créé par Craig Reynolds en 1986, simule le comportement collectif d'oiseaux ou de poissons. L'intérêt majeur pour la médiation scientifique est d'illustrer le concept d'émergence : un comportement global complexe et harmonieux (la nuée) qui n'est pas dicté par un leader, mais qui résulte de l'application de règles locales très simples par chaque individu.

4.2 – Modélisation : Les trois règles fondamentales

Chaque agent (un "Boid") est un objet physique possédant une position, un vecteur vitesse et une accélération. À chaque itération, trois forces sont calculées :

- 1- La Séparation : Pour éviter les collisions. L'agent calcule la distance avec ses voisins. S'ils sont trop proches, une force de répulsion est générée.

$$\text{- Formule simplifiée : } F_{sep} = \sum \frac{pos_{moi} - pos_{voisin}}{distance^2} .$$

- 2- L'Alignement : Pour l'harmonie du groupe. L'agent essaie d'ajuster sa direction sur la moyenne de la direction de ses voisins.

- 3- La Cohésion : Pour rester groupés. L'agent est attiré vers le "centre de masse" (la position moyenne) de ses voisins.

L'accélération finale est la somme pondérée de ces trois vecteurs. Dans notre démonstrateur, l'utilisateur peut modifier ces poids (coefficients) en temps réel pour observer l'impact sur la nuée.

4.3 – Optimisation technique et gestion des limites (Le défi du $O(n^2)$)\

L'une des problématiques majeures rencontrées lors du développement du simulateur de Boids est celle de la montée en charge. Dans un modèle naïf, chaque agent doit calculer sa distance par rapport à tous les autres agents pour déterminer son voisinage. Pour un nombre n d'agents, nous effectuons n^2 comparaisons à chaque frame. À 60 images par seconde, avec 500 Boids, cela représente 15 millions de calculs de distance par seconde.

La perception limitée : Pour optimiser le rendu sans sacrifier le réalisme, nous avons implémenté un rayon de perception. Un agent ne réagit qu'aux voisins situés dans un cercle de rayon R .

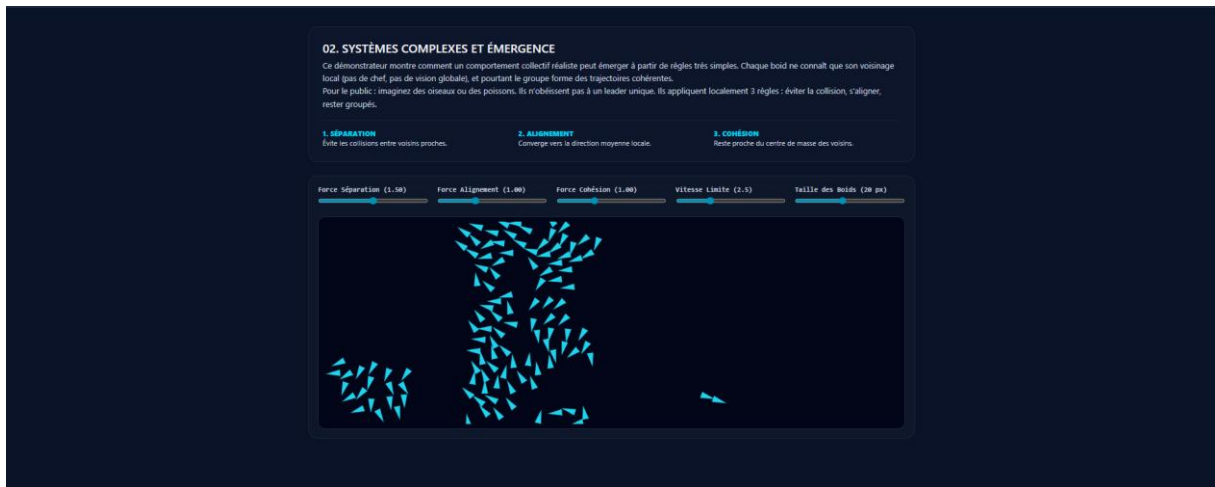
- Gestion des bords : Nous avons implémenté un espace torique (les agents qui sortent à droite réapparaissent à gauche), ce qui évite les effets d'accumulation sur les bords de l'écran et maintient une densité constante de la population.
- Évitement d'obstacles : Une quatrième force a été ajoutée : la force de répulsion environnementale. Lorsqu'un agent détecte un obstacle (forme géométrique fixe sur le Canvas), il génère un vecteur de braquage (steering force) proportionnel à l'inverse du carré de la distance, illustrant ainsi comment un groupe peut contourner un danger sans se désagréger.

4.4 – Analyse expérimentale et réglages du comportement

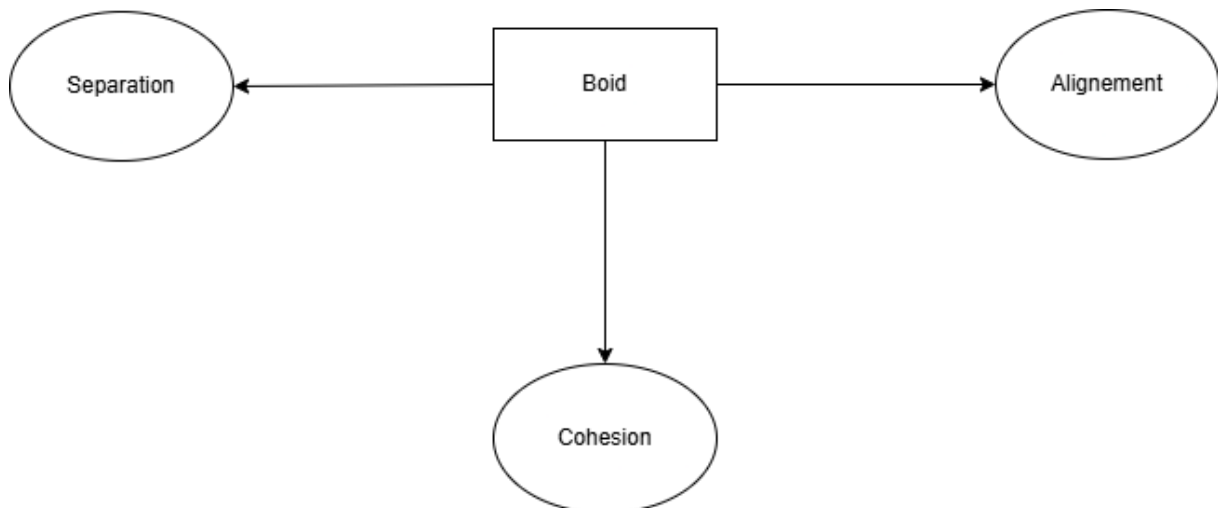
Le démonstrateur permet de comparer différents réglages pour observer des phénomènes de transition de phase :

- Comportement de "Nuage" : Une cohésion forte et un alignement faible créent un groupe compact mais désordonné, semblable à un essaim d'insectes.

- Comportement de "Migration" : Un alignement prédominant entraîne le groupe dans



une direction unique et rectiligne.



- Comportement de "Vortex" : En jouant sur les délais de mise à jour des vecteurs, on peut observer l'émergence de mouvements circulaires complexes.

4.5 – Visuel indispensable

Capture montrant le démonstrateur de Boids avec les différents paramètres ajustable en temps réel.

Ce schéma nous montre le fonctionnement d'un Boid qui a 3 paramètres attribue :

- Séparation : dirigé vers l'opposé des voisins proche.
- Alignement : dirigé dans la moyenne des directions des voisins.
- Cohésion : dirigé vers le centre de masse des voisins.

Démonstrateur 3 : Réseaux Sociaux

5.1 – Théorie des graphes et modélisation sociale

Le troisième démonstrateur traite de la propagation de l'information (ou de virus) au sein d'un réseau structuré [cite : 2]. Le réseau est modélisé mathématiquement par un graphe $G = (V, E)$:

- V (sommets) : Les individus ou comptes sociaux
- E (Arêtes) : Les interactions, amitiés ou liens d'influence. En informatique, ce graphe est stocké sous forme d'une **liste d'adjacence**, structure de donnée optimisée pour parcourir rapidement les voisins d'un nœud lors d'une diffusion.

5.2 – Algorithme et modèles de propagation

Pour illustrer la diversité des phénomènes sociaux, nous avons implémenté trois modèles distincts [cite : 2] :

5.2.1 – Diffusion déterministe vs Probabiliste

- Modèle déterministe : Si le nœud A est infecté, tous ses voisins directs le deviennent à l'étape suivante. C'est le modèle de la rumeur parfaite.
- Modèle probabiliste : Chaque lien possède une probabilité P de transmission. Cela permet de simuler la résistance à l'information ou l'efficacité relative d'un message. L'utilisateur peut modifier ce curseur P pour observer le "seuil de criticité" : en dessous d'un certain taux, l'information s'éteint d'elle-même.

5.2.2 – Le modèle SIR (Susceptible, Infected, Recovered)

Inspiré de l'épidémiologie, ce modèle ajoute une dimension temporelle :

- 1- S (Sain) : Le nœud peut recevoir l'information.
- 2- I (Infecté) : Le nœud propage l'information.
- 3- R (Rétabli/Immunisé) : Après un certain temps, le nœud cesse de propager et ne peut plus être infecté. C'est le modèle le plus riche visuellement, car il permet de voir des "vagues" de propagation traverser le graphe.

5.3 – Interactivité et visualisation "Force-Directed"

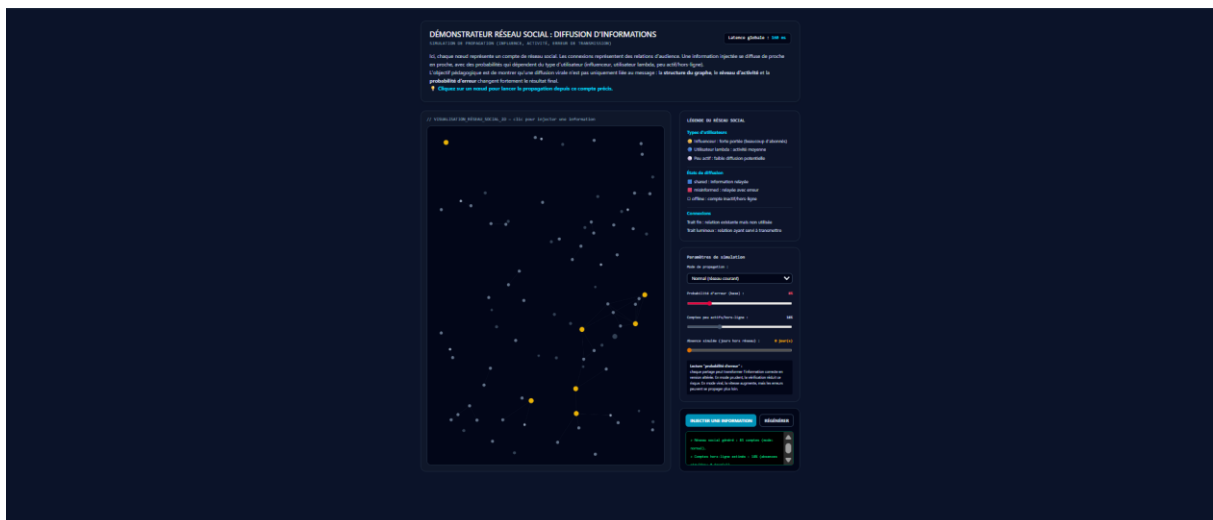
Pour rendre le graphe lisible, nous avons implémenté un algorithme de positionnement par forces (Force-Directed Layout). Les nœuds se repoussent entre eux (force électrostatique) tandis que les arêtes agissent comme des ressorts qui les attirent.

- Résultat : Le graphe s'auto-organise à l'écran. Les communautés se regroupent, et les nœuds centraux (hubs) se placent naturellement au milieu de la scène, rendant la structure du réseau immédiatement compréhensible pour le public.

5.4 – Analyse de l'influence de la structure

L'un des objectifs majeurs est de montrer l'influence de la topologie sur la vitesse de diffusion [cite : 2].

- Les Influenceurs (Hubs) : Si un nœud possède un degré très élevé (beaucoup de

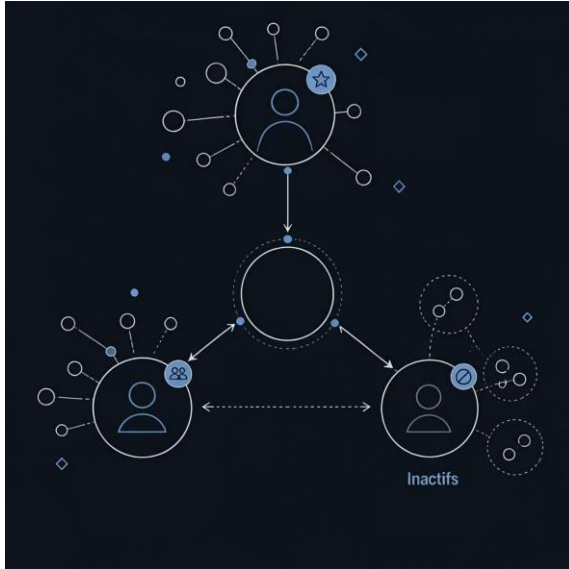


liens), son infection provoque une explosion immédiate de la diffusion.

- Les Clusters : On observe que l'information peut rester "piégée" dans un groupe très dense de nœuds avant de réussir (ou non) à franchir un "pont" vers une autre partie du réseau.

5.5 – Visuel indispensable

Sur cette capture on peut voir le démonstrateur concernant les réseaux sociaux. Avec la légende à droite et les différents paramètres modifiable.



Ce schéma nous montre les différents types d'utilisateurs et comment ils interagissent dans le réseau. On peut donc voir que l'influenceur a une puissance de partage supérieur aux deux autres.

Analyse Critique et Perspectives

6.1 – Bilan technique et limites rencontrées

Le projet a permis de valider la viabilité des technologies web pour la médiation scientifique. L'utilisation du Canvas HTML5 a offert la fluidité nécessaire, et Vue.js a simplifié la création d'une interface interactive complexe. Cependant, des limites ont été identifiées :

- Performances CPU : JavaScript étant mono-thread, les simulations de Boids saturent au-delà de 400 agents sur des machines peu puissantes.
- Précision algorithmique : Pour privilégier le visuel, certains calculs de collision ont été simplifiés, ce qui pourrait être affiné pour une simulation plus rigoureuse.

6.2 – Ouvertures et améliorations futures

Pour transformer ce prototype en outil industriel, plusieurs pistes sont envisagées :

- 1- Partitionnement spatial (QuadTree) : Pour le modèle de Boids, découper l'espace en zones permettrait de passer d'une complexité $O(n^2)$ à $O(n \log n)$ autorisant ainsi des milliers d'agents simultanés.
- 2- Web Workers : Déporter les calculs lourds (Sudoku, Graphes) dans des threads séparés pour ne jamais geler l'interface utilisateur.
- 3- WebAssembly (WASM) : Réécrire les moteurs de calcul en C++ ou Rust pour obtenir des performances proches du natif.

Conclusion

Le projet de conception de démonstrateurs algorithmiques interactifs a atteint ses objectifs de médiation scientifique [cite : 2]. En transformant des concepts abstraits (CSP, émergence, théorie des graphes) en expériences visuelles et interactives, nous avons créé un pont entre la rigueur académique et l'intuition du grand public.

Ce travail a permis de mobiliser l'ensemble des compétences acquises durant la Licence Informatique : de la maîtrise des structures de données complexes à l'analyse de la complexité algorithmique, en passant par le développement d'interfaces modernes. Au-delà de l'aspect technique, ce projet souligne l'importance du rôle de l'informaticien comme communicant, capable de vulgariser ses outils pour les rendre accessibles à tous.

Source bibliographique

Ouvrages et articles de référence :

- Sur l'Intelligence Artificielle et les CSP (Sudoku) :
 - Russell, S. J., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach*. Pearson Education. (Section sur le *Constraint Satisfaction Problem* et le *Backtracking*).
- Sur les Systèmes Multi-agents (Boids) :
 - Reynolds, C. W. (1987). "Flocks, herds and schools: A distributed behavioral model". *ACM SIGGRAPH Computer Graphics*, 21(4), 25-34.

- Sur la Théorie des Graphes et Réseaux Sociaux :
 - Easley, D., & Kleinberg, J. (2010). *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press.
- Sur le Développement Web (Techniques) :
 - Mozilla Developer Network (MDN). (2026). *Canvas API - Guide de référence*. https://developer.mozilla.org/fr/docs/Web/API/Canvas_API
 - Vue.js Team. (2026). *Guide officiel de la Composition API*. <https://vuejs.org/guide/>

Annexes

9.1 – Annexe A : Guide d'installation et Manuel utilisateur

9.1.1 – Prérequis techniques

Avant de lancer les démonstrateurs, assurez-vous que l'environnement de développement suivant est configuré sur votre machine :

- Node.js : Version 18.0.0 ou supérieure.
- Gestionnaire de paquets : npm (installé avec Node.js).
- Terminal : Un accès à l'invite de commande (CMD/PowerShell sur Windows, Terminal sur macOS/Linux).

9.1.2 – Installation des dépendances

Le projet repose sur le framework **Vue.js** et nécessite l'installation des bibliothèques nécessaires. Dans le dossier racine du projet, exécutez la commande suivante :

npm install

Cette commande génère le dossier `node_modules/` contenant toutes les dépendances requises pour la simulation des algorithmes.

9.1.3 – Scripts de lancement automatisés

Pour simplifier l'utilisation sur les différents systèmes d'exploitation, deux scripts ont été développés pour automatiser l'installation et le lancement du serveur de développement :

- Un script bat pour les utilisateurs windows
- Un script shell pour les utilisateurs MacOS/Linux

9.1.4 – Guide d'utilisation rapide

Une fois le projet lancé, l'interface s'ouvre dans votre navigateur par défaut.

1. Menu latéral : Permet de basculer entre les trois démonstrateurs (Sudoku, Boids, Réseaux).
2. Curseurs interactifs : Chaque démonstrateur possède des paramètres modifiables (vitesse, nombre d'agents, probabilité d'infection).
3. Visualisation : Les calculs sont effectués en temps réel dans le Canvas HTML5. Si le navigateur semble ralentir, réduisez le nombre d'agents (pour les Boids) ou la vitesse de l'algorithme (pour le Sudoku).

9.1.5 – Dépannage (Troubleshooting)

- Erreur "npm command not found" : Node.js n'est pas installé ou n'est pas dans votre PATH système. Veuillez le réinstaller depuis nodejs.org.

- Port 5173 déjà utilisé : Si le serveur ne démarre pas, vérifiez qu'une autre instance de Vite/Vue n'est pas déjà lancée en arrière-plan et fermez-la.

9.2 – Annexe B : Glossaire Technique

9.2.1 – Algorithmique et Théorie des Graphes

Algorithme de Backtracking : Méthode de résolution de problèmes par recherche exhaustive dans un arbre de décision. Si une impasse est rencontrée (violation d'une contrainte), l'algorithme "revient en arrière" pour explorer une autre branche.

CSP (Constraint Satisfaction Problem) : Problème mathématique consistant à trouver des valeurs pour un ensemble de variables, telles que toutes les contraintes données soient satisfaites.

Élagage (Pruning) : Technique consistant à supprimer des branches inutiles d'un arbre de recherche pour réduire l'espace des solutions à explorer et accélérer le calcul.

Graphe : Structure de données composée d'un ensemble de **sommets** (ou nœuds) reliés par des **arêtes** (ou liens).

Heuristique : Stratégie de recherche non optimale mais efficace, utilisée pour accélérer la résolution d'un problème complexe (ex : choix de la variable la plus contrainte).

Modèle SIR : Modèle épidémiologique compartimental divisant une population en trois états : **Susceptible** (sain), **Infecté** (porteur), **Rétabli** (immunisé).

Topologie : Étude de la disposition et de la connectivité des éléments d'un réseau.

9.2.2 – Systèmes Multi-Agents et Simulation

Agent Autonome : Entité logicielle capable de percevoir son environnement et de prendre des décisions basées sur des règles locales simples sans intervention centrale.

Boids : Contraction de "Bird-oid objects". Modèle informatique simulant le comportement de vol coordonné de groupes d'oiseaux ou de poissons.

Comportement Émergent : Phénomène où des entités simples interagissant localement produisent un comportement global complexe et imprévu, sans planificateur central.

Règles de Reynolds : Ensemble de trois règles fondamentales régissant les Boids :

- *Séparation* : Éviter les collisions avec les voisins.
- *Alignement* : Diriger sa vitesse vers la moyenne de celle des voisins.
- *Cohésion* : Se diriger vers le centre de masse des voisins.

9.2.3 – Développement Web et Rendu Graphique

Canvas API : Élément HTML5 permettant de générer des graphismes dynamiques et complexes en temps réel via des scripts (généralement JavaScript).

DOM (Document Object Model) : Interface de programmation représentant la structure d'un document HTML. C'est l'arbre sur lequel le framework s'appuie pour manipuler l'interface.

Framework : Ensemble de bibliothèques et de structures de code (ici Vue.js) facilitant le développement d'applications web.

Réactivité (State Management) : Mécanisme par lequel l'interface utilisateur se met à jour automatiquement dès que les données sous-jacentes (le "State") sont modifiées par l'algorithme.

Thread Principal (Main Thread) : Fil d'exécution unique responsable de l'affichage et de la gestion des interactions utilisateur dans un navigateur. Une opération trop longue dans ce thread peut bloquer l'interface.

9.3 – Annexe C : Code

```
/**
 * resoudreBrut() "resoudre": Unknown word.
 * -----
 * Solveur 1 : backtracking naïf. "Solveur": Unknown word.
 * Parcours linéaire des cases vides + test 1.9. "Parcours": Unknown word.
 */
const resoudreBrut = async () => { "resoudre": Unknown word.
  const recherche = async () => {
    if (signalAnnulation.value) return false

    for (let r = 0; r < 9; r++) {
      for (let c = 0; c < 9; c++) {
        if (grilleBrute.value[r][c] === 0) {
          for (let num = 1; num <= 9; num++) {
            essaisBrut.value++ "essais": Unknown word.

            if (estValide(grilleBrute.value, r, c, num)) { "Valide": Misspelled word.
              grilleBrute.value[r][c] = num
              await attendre() "attendre": Unknown word.

              if (await recherche()) return true

              grilleBrute.value[r][c] = 0
              await attendre() "attendre": Unknown word.
            }
          }
        }
      }
    }
    return false
  }
  return true
}

await recherche()
}
```

Fonction de backtracking

```
const updateBoids = () => { "Boids": Unknown word.
  const canvas = canvasRef.value
  if (!canvas) return
  const w = canvas.width
  const h = canvas.height

  boids.value.forEach(b => { "boids": Unknown word.
    let closeDx = 0, closeDy = 0
    let avgVx = 0, avgVy = 0, alignmentNeighbors = 0
    let avgX = 0, avgY = 0, cohesionNeighbors = 0

    boids.value.forEach(other => { "boids": Unknown word.
      if (b === other) return
      const dx = b.x - other.x
      const dy = b.y - other.y
      const dist = Math.hypot(dx, dy)

      if (dist < separationDist.value) {
        closeDx += dx
        closeDy += dy
      }
      if (dist < alignmentDist.value) {
        avgVx += other.vx
        avgVy += other.vy
        alignmentNeighbors++
      }
      if (dist < cohesionDist.value) {
        avgX += other.x
        avgY += other.y
        cohesionNeighbors++
      }
    })

    b.vx += closeDx * separationFactor.value * 0.05
    b.vy += closeDy * separationFactor.value * 0.05

    if (alignmentNeighbors > 0) {
      avgVx /= alignmentNeighbors
      avgVy /= alignmentNeighbors
      b.vx += (avgVx - b.vx) * alignmentFactor.value * 0.05
      b.vy += (avgVy - b.vy) * alignmentFactor.value * 0.05
    }
  })
}
```

```
if (cohesionNeighbors > 0) {
  avgX /= cohesionNeighbors
  avgY /= cohesionNeighbors
  b.vx += (avgX - b.x) * cohesionFactor.value * 0.01
  b.vy += (avgY - b.y) * cohesionFactor.value * 0.01
}

const speed = Math.hypot(b.vx, b.vy)
if (speed > maxSpeed.value) {
  b.vx = (b.vx / speed) * maxSpeed.value
  b.vy = (b.vy / speed) * maxSpeed.value
}

b.x += b.vx
b.y += b.vy

if (b.x < 0) b.x = w
if (b.x > w) b.x = 0
if (b.y < 0) b.y = h
if (b.y > h) b.y = 0
)
```

Paramètres alignement/cohésion/séparation des boids